UNIDAD 8
PROGRAMACIÓN DE BASES DE DATOS EN MYSQL

FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

Unidad Didáctica	Unidad Didáctica 8: Programación de bases de datos: Procedimientos, funciones y triggers en MySQL
Resultados de Aprendizaje	**RA6:** Programa procedimientos almacenados, funciones y disparadores (triggers) analizando su funcionamiento e integrando lógica procedimental en el SGBD para garantizar la automatización, integridad y rendimiento de los datos.
Criterios de Evaluación	**CE6.a** (Se han identificado los lenguajes de programación disponibles), **CE6.b** (Se han definido procedimientos con paso de parámetros), **CE6.c** (Se han creado funciones), **CE6.d** (Se han utilizado estructuras de control de flujo), **CE6.e** (Se han utilizado cursores), **CE6.f** (Se ha realizado el control de excepciones), **CE6.g** (Se han creado disparadores), **CE6.h** (Se han depurado y probado los subprogramas creados).
Tiempo Estimado	24 Horas (Distribuidas en 4 semanas de sesiones teórico-prácticas orientadas a la optimización de la lógica del servidor).

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. PROCEDIMIENTOS ALMACENADOS

2.1 Sintaxis general y uso de DELIMITER

2.2 Gestión de Parámetros: IN, OUT y INOUT

2.3 Ámbito de Variables Locales y Asignaciones (DECLARE, SET)

3. FUNCIONES ALMACENADAS

3.1 Diferencias con procedimientos y restricciones de retorno

3.2 Modificadores de comportamiento de funciones en MySQL

4. ESTRUCTURAS DE CONTROL DE FLUJO

4.1 Condicionales (IF-THEN-ELSE y CASE)

4.2 Estructuras Iterativas (LOOP, REPEAT y WHILE)

5. MANEJO DE EXCEPCIONES Y ERRORES EN MYSQL

5.1 El uso de DECLARE... HANDLER

6. CURSORES

6.1 Ciclo de vida de un cursor

7. DISPARADORES (TRIGGERS)

7.1 Lógica de ejecución: BEFORE/AFTER e INSERT/UPDATE/DELETE

7.2 Pseudotablas OLD y NEW y su aplicación en la integridad de datos

8. ENLACES DE REFERENCIA Y DOCUMENTACIÓN OFICIAL

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

La administración moderna de bases de datos a nivel de infraestructura no se limita a ejecutar consultas DML de manera interactiva. Para garantizar la eficiencia de los sistemas de red, la seguridad de la información y la automatización de procesos, los administradores de sistemas deben ser capaces de almacenar lógica programada directamente en el servidor SGBD.

Esta unidad se conecta directamente con:

- **GBD (Unidad 5 y 6):** Emplea de manera avanzada comandos DML de consultas simples, agrupaciones y composiciones de tablas (JOIN) dentro de la lógica del servidor.
- **ISO (Implantación de Sistemas Operativos):** Se relaciona con la automatización mediante scripting. Un *SysAdmin* puede encapsular flujos complejos dentro de procedimientos en el servidor de base de datos y activarlos desde tareas programadas del sistema operativo (*cron jobs* o *Windows Task Scheduler*).

- **SAD (Seguridad y Alta Disponibilidad) / GBD (Seguridad):** El uso de rutinas almacenadas permite retirar permisos directos de lectura/escritura (SELECT, INSERT, UPDATE) sobre las tablas de producción de la base de datos **Jardinería**, otorgando únicamente privilegios de ejecución (EXECUTE) sobre procedimientos. Esto mitiga de forma casi absoluta los ataques de **inyección SQL** (*SQL Injection*).

2. PROCEDIMIENTOS ALMACENADOS

Un procedimiento almacenado es un conjunto de sentencias SQL preparadas y guardadas en el SGBD para poder ser reutilizadas repetidamente de forma eficiente [7].

2.1 Sintaxis general y uso de DELIMITER

Por defecto, el SGBD interpreta el carácter punto y coma (;) como el terminador de una sentencia SQL. Al definir un bloque de código procedimental que contiene múltiples sentencias finalizadas con ;, necesitamos indicarle al motor de MySQL que cambie temporalmente este símbolo para que compile todo el bloque como una sola instrucción. Esto se logra con la directiva DELIMITER [7].

```
-- Cambiamos el terminador por defecto a doble barra inclinada
DELIMITER //
DROP PROCEDURE IF EXISTS ejemplo_basico//
CREATE PROCEDURE ejemplo_basico()
BEGIN
-- El terminador temporal es //, el ; interno no interrumpe la
compilación
SELECT 'Procedimiento almacenado de Jardinería inicializado
correctamente' AS mensaje;
END//
-- Restauramos el delimitador estándar
DELIMITER ;
-- Invocación del procedimiento
CALL ejemplo_basico();
```

2.2 Gestión de Parámetros: IN, OUT y INOUT

MySQL permite especificar tres tipos de dirección para los parámetros de entrada y salida de un procedimiento almacenado [7, 11.2]:

- **IN (Entrada):** Es el valor que se pasa al procedimiento al llamarlo. Se trata de un parámetro de solo lectura dentro de la rutina (es el comportamiento por defecto si se omite la palabra clave) [7].
- **OUT (Salida):** Almacena el valor calculado por el procedimiento y lo devuelve al entorno de llamada, generalmente asignándole a una variable de usuario (con prefijo @) [7, 11.2].
- **INOUT (Entrada/Salida):** Se inicia al llamar al procedimiento, puede ser modificado por la rutina y devuelve el nuevo valor de vuelta al llamante [11.2].

A continuación se muestra la implementación de un procedimiento para actualizar el límite de crédito de un cliente de la base de datos **Jardinería** utilizando un parámetro IN, y otro procedimiento procedimental para calcular estadísticas de un cliente con parámetros OUT.

Ejemplo de parámetro IN (Actualización de Límite de Crédito):

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_actualizar_limite_credito//
CREATE PROCEDURE sp_actualizar_limite_credito(
IN p_codigo_cliente INT,
IN p_porcentaje_incremento DECIMAL(5,2)
)
BEGIN
-- Modifica el límite de crédito aplicando el incremento
porcentual
UPDATE cliente
SET limite_credito = limite_credito * (1 + (p_porcentaje_incremento /
100))
WHERE codigo_cliente = p_codigo_cliente;
END//
DELIMITER ;
-- Probamos el procedimiento con el cliente 101 incrementando un
15.5% su crédito
CALL sp_actualizar_limite_credito(101, 15.50);
```

Ejemplo de parámetros OUT (Extracción de estadísticas de ventas):

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_obtener_estadisticas_cliente//
CREATE PROCEDURE sp_obtener_estadisticas_cliente(
IN p_codigo_cliente INT,
OUT p_total_pedidos INT,
```

```
OUT p_total_pagado DECIMAL(15,2)
)
BEGIN
-- Contamos el número de pedidos realizados por el cliente
SELECT COUNT(*) INTO p_total_pedidos
FROM pedido
WHERE codigo_cliente = p_codigo_cliente;
-- Calculamos el acumulado total de los pagos registrados por este
cliente
SELECT COALESCE(SUM(total), 0) INTO p_total_pagado
FROM pago
WHERE codigo_cliente = p_codigo_cliente;
END//
DELIMITER ;
-- Invocación utilizando variables de sesión de MySQL para recoger
las salidas
CALL sp_obtener_estadisticas_cliente(101, @pedidos, @pagado);
-- Visualizamos los resultados
SELECT @pedidos AS pedidos_realizados, @pagado AS
total_euros_pagados;
```

2.3 Ámbito de Variables Locales y Asignaciones (DECLARE, SET)

Las variables internas deben declararse obligatoriamente al principio de un bloque BEGIN ... END utilizando la instrucción DECLARE [11.2]. Tras su declaración, se les puede dar un valor inicial con la cláusula DEFAULT o modificarse utilizando SET u obteniendo datos mediante la cláusula INTO dentro de un SELECT.

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_demo_variables_jardineria//
CREATE PROCEDURE sp_demo_variables_jardineria(
IN p_codigo_oficina VARCHAR(10)
)
BEGIN
-- Declaración de variables locales tipadas
DECLARE v_ciudad_oficina VARCHAR(50);
DECLARE v_cantidad_empleados INT DEFAULT 0;
-- Asignación directa por consulta DML
SELECT ciudad INTO v_ciudad_oficina
FROM oficina
WHERE codigo_oficina = p_codigo_oficina;
-- Asignación por recuento
```

```
SELECT COUNT(*) INTO v_cantidad_empleados
FROM empleado
WHERE codigo_oficina = p_codigo_oficina;
-- Presentación de resultados
SELECT v_ciudad_oficina AS ciudad, v_cantidad_empleados AS
num_empleados_activos;
END//
DELIMITER ;
CALL sp_demo_variables_jardineria('BCN-ES');
```

3. FUNCIONES ALMACENADAS

Una función almacenada es una rutina que toma parámetros de entrada, ejecuta lógica procedural y **devuelve un único valor escalar** al entorno de llamada.

3.1 Diferencias con procedimientos y restricciones de retorno

A diferencia de los procedimientos almacenados, las funciones:

1. Deben especificar la cláusula RETURNS en su cabecera para definir el tipo de datos del resultado.
2.
```
Contienen obligatoriamente al menos una sentencia RETURN que finaliza
la ejecución de la función.
```
3.
```
No se invocan con la sentencia CALL, sino que se pueden utilizar
directamente dentro de sentencias DML (como parte de un SELECT, WHERE u
ORDER BY), optimizando la legibilidad de las consultas complejas.
```

3.2 Modificadores de comportamiento de funciones en MySQL

Al compilar funciones en MySQL con el motor InnoDB activo, es obligatorio declarar sus características de acceso a datos para que el planificador y el motor de replicación trabajen con seguridad lógica [1.9.3]:

- DETERMINISTIC: El resultado devuelto será idéntico siempre que se proporcionen los mismos parámetros de entrada.
- NOT DETERMINISTIC: El resultado puede cambiar aun pasando los mismos parámetros (por ejemplo, funciones que consulten la fecha actual o generen números aleatorios).
- READS SQL DATA: Indica que la rutina realizará consultas de lectura (SELECT) sobre tablas de la base de datos [1.9.3].
- MODIFIES SQL DATA: Indica que la rutina ejecutará modificaciones (INSERT, UPDATE, DELETE).

Ejemplo (Calcular deuda de un cliente de Jardinería):

Implementación de una función procedimental que calcula la diferencia acumulada entre el total facturado en pedidos al cliente y los pagos que este ha liquidado.

```
DELIMITER //
DROP FUNCTION IF EXISTS fn_calcular_deuda_cliente//
CREATE FUNCTION fn_calcular_deuda_cliente(
p_codigo_cliente INT
)
RETURNS DECIMAL(15,2)
READS SQL DATA
BEGIN
DECLARE v_total_pedidos DECIMAL(15,2) DEFAULT 0.00;
DECLARE v_total_pagado DECIMAL(15,2) DEFAULT 0.00;
DECLARE v_deuda DECIMAL(15,2) DEFAULT 0.00;
-- 1. Calcular el coste total acumulado de todos los pedidos
detallados del cliente
SELECT COALESCE(SUM(dp.cantidad * dp.precio_unidad), 0.00) INTO
v_total_pedidos
FROM pedido p
JOIN detalle_pedido dp ON p.codigo_pedido = dp.codigo_pedido
WHERE p.codigo_cliente = p_codigo_cliente;
-- 2. Calcular la suma de todos los pagos realizados por el
cliente
SELECT COALESCE(SUM(total), 0.00) INTO v_total_pagado
FROM pago p
WHERE p.codigo_cliente = p_codigo_cliente;
-- 3. Restar para obtener la deuda activa
SET v_deuda = v_total_pedidos - v_total_pagado;
RETURN v_deuda;
END//
```

```
DELIMITER ;
-- Invocación directa dentro de una consulta DML SELECT estándar de
Jardinería
SELECT
codigo_cliente,
nombre_cliente,
limite_credito,
fn_calcular_deuda_cliente(codigo_cliente) AS saldo_deudor_activo
FROM cliente
WHERE ciudad = 'Madrid';
```

4. ESTRUCTURAS DE CONTROL DE FLUJO

La programación procedural de bases de datos permite redirigir el orden de procesamiento de las sentencias utilizando estructuras lógicas avanzadas de decisión y repetición.

4.1 Condicionales (IF-THEN-ELSE y CASE)

Permiten ejecutar diferentes fragmentos de código basándose en la evaluación de una condición booleana.

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_evaluar_riesgo_cliente//
CREATE PROCEDURE sp_evaluar_riesgo_cliente(
IN p_codigo_cliente INT,
OUT p_nivel_riesgo VARCHAR(30)
)
BEGIN
DECLARE v_deuda DECIMAL(15,2);
DECLARE v_limite DECIMAL(15,2);
-- Ejecutamos la llamada a la función previamente programada
SET v_deuda = fn_calcular_deuda_cliente(p_codigo_cliente);
-- Extraemos el límite de crédito asignado
SELECT limite_credito INTO v_limite
FROM cliente
WHERE codigo_cliente = p_codigo_cliente;
-- Bifurcación condicional compleja IF-THEN-ELSE
IF v_deuda <= 0.00 THEN
SET p_nivel_riesgo = 'EXCELENTE (SIN DEUDA)';
ELSEIF v_deuda > v_limite THEN
SET p_nivel_riesgo = 'CRÍTICO: LÍMITE EXCEDIDO';
ELSEIF v_deuda > (v_limite * 0.75) THEN
SET p_nivel_riesgo = 'ATENCIÓN: DEUDA ELEVADA (>75%)';
```

```
ELSE
SET p_nivel_riesgo = 'RIESGO BAJO / CONTROLADO';
END IF;
END//
DELIMITER ;
CALL sp_evaluar_riesgo_cliente(101, @riesgo);
SELECT @riesgo AS estado_riesgo_financiero;
```

4.2 Estructuras Iterativas (LOOP, REPEAT y WHILE)

MySQL proporciona estructuras nativas para realizar bucles lógicos [11.2]. En entornos de producción de administración de sistemas, es indispensable etiquetar estos bucles para gestionar flujos de control complejos mediante sentencias de escape como LEAVE (equivalente a *break*) o ITERATE (equivalente a *continue*) [12.1.8.2].

A continuación se muestra un ejemplo comparativo de implementación de secuencias mediante bucles procedurales aplicados a un escenario administrativo para poblar una tabla de auditoría con rangos numéricos.

```
-- Creamos una tabla auxiliar para almacenar históricos o rangos
CREATE TABLE IF NOT EXISTS auxiliar_rangos (
iteracion INT UNSIGNED PRIMARY KEY,
mensaje_logs VARCHAR(100)
);
-- Implementación usando WHILE: Verifica la condición antes de entrar
al bucle
DELIMITER //
DROP PROCEDURE IF EXISTS sp_llenar_while//
CREATE PROCEDURE sp_llenar_while(IN p_tope INT)
BEGIN
DECLARE v_contador INT DEFAULT 1;
TRUNCATE TABLE auxiliar_rangos;
WHILE v_contador <= p_tope DO
INSERT INTO auxiliar_rangos VALUES (v_contador, CONCAT('Registro
automatizado por bucle WHILE: ', v_contador));
SET v_contador = v_contador + 1;
END WHILE;
END//
-- Implementación usando REPEAT: Ejecuta el bloque y luego evalúa la
condición de salida
DROP PROCEDURE IF EXISTS sp_llenar_repeat//
CREATE PROCEDURE sp_llenar_repeat(IN p_tope INT)
BEGIN
```

```
DECLARE v_contador INT DEFAULT 1;
TRUNCATE TABLE auxiliar_rangos;
REPEAT
INSERT INTO auxiliar_rangos VALUES (v_contador, CONCAT('Registro
automatizado por bucle REPEAT: ', v_contador));
SET v_contador = v_contador + 1;
UNTIL v_contador > p_tope
END REPEAT;
END//
-- Implementación usando LOOP: Bucle infinito controlado manualmente
mediante LEAVE e ITERATE
DROP PROCEDURE IF EXISTS sp_llenar_loop//
CREATE PROCEDURE sp_llenar_loop(IN p_tope INT)
BEGIN
DECLARE v_contador INT DEFAULT 0;
TRUNCATE TABLE auxiliar_rangos;
mi_bucle: LOOP
SET v_contador = v_contador + 1;
-- Condición de parada de emergencia
IF v_contador > p_tope THEN
LEAVE mi_bucle;
END IF;
-- Saltarse números pares en la inserción (Uso de ITERATE)
IF MOD(v_contador, 2) = 0 THEN
ITERATE mi_bucle;
END IF;
INSERT INTO auxiliar_rangos VALUES (v_contador, CONCAT('Registro
impar automatizado por bucle LOOP: ', v_contador));
END LOOP mi_bucle;
END//
DELIMITER ;
CALL sp_llenar_loop(5);
SELECT * FROM auxiliar_rangos;
```

5. MANEJO DE EXCEPCIONES Y ERRORES EN MYSQL

El código procedimental que ejecuta cambios DML puede fallar debido a violaciones de integridad referencial, registros duplicados en claves primarias o caídas del sistema. Para evitar que la transacción completa aborte de forma incontrolada, MySQL ofrece el uso de manejadores de error declarativos [1.9].

5.1 El uso de DECLARE... HANDLER

La sintaxis general para registrar un manejador de eventos excepcionales es la siguiente:

```
DECLARE {CONTINUE | EXIT} HANDLER FOR condicion_error sentencia;
```

Donde:

- **CONTINUE:** Ejecuta la sentencia correctora asociada y continúa con el flujo lógico del bloque procedimental tras la instrucción que generó la alerta [1.9].
- **EXIT:** Detiene la ejecución del bloque actual (BEGIN ... END) de manera inmediata tras realizar las acciones de resolución de errores establecidas [1.9].
- **condicion_error:** Puede ser un código de error numérico interno del SGBD, un estándar SQLSTATE de la industria o palabras clave genéricas como SQLEXCEPTION, SQLWARNING o NOT FOUND [1.9].

Ejemplo de registro de pago de cliente seguro ante claves duplicadas:

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_registrar_pago_seguro//
CREATE PROCEDURE sp_registrar_pago_seguro(
IN p_codigo_cliente INT,
IN p_forma_pago VARCHAR(40),
IN p_id_transaccion VARCHAR(50),
IN p_fecha_pago DATE,
IN p_total DECIMAL(15,2),
OUT p_resultado_operacion VARCHAR(100)
)
BEGIN
-- Manejador específico para el Error 1062 de MySQL (Entrada
duplicada en clave primaria)
DECLARE EXIT HANDLER FOR 1062
BEGIN
SET p_resultado_operacion = CONCAT('RECHAZADO: El ID de transacción
"', p_id_transaccion, '" ya existe en el histórico de pagos.');
END;
-- Manejador general para cualquier otro tipo de error de restricción
o datos (Llave foránea incorrecta, etc.)
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
```

```
SET p_resultado_operacion = 'RECHAZADO: Error general de integridad
de base de datos.';
END;
-- Intentamos la inserción en la tabla de Jardinería
INSERT INTO pago (codigo_cliente, forma_pago, id_transaccion,
fecha_pago, total)
VALUES (p_codigo_cliente, p_forma_pago, p_id_transaccion,
p_fecha_pago, p_total);
SET p_resultado_operacion = 'OPERACIÓN COMPLETADA CON ÉXITO';
END//
DELIMITER ;
-- Intentamos registrar una transacción segura
CALL sp_registrar_pago_seguro(1, 'Transferencia', 'TR-REG-001',
'2026-06-19', 450.50, @estado);
SELECT @estado;
-- Volvemos a llamar al procedimiento con la misma transacción para
provocar la excepción de duplicidad controlada
CALL sp_registrar_pago_seguro(1, 'Transferencia', 'TR-REG-001',
'2026-06-19', 450.50, @estado);
SELECT @estado;
```

6. CURSORES

Un cursor es una estructura procedimental en memoria que permite iterar y procesar, registro por registro, el conjunto de filas devuelto por una consulta SELECT.

6.1 Ciclo de vida de un cursor

El uso de un cursor sigue estrictamente un proceso secuencial de 4 pasos [1.8.7]:

1. **DECLARE:** Define el cursor vinculándolo a una consulta SQL selectiva. Esta instrucción debe colocarse después de declarar todas las variables locales pero antes de los manejadores de excepciones.
2. **OPEN:** Ejecuta la consulta SQL e inicializa el puntero en memoria, dejándolo preparado para la extracción.
3. **FETCH:** Extrae secuencialmente los valores de la fila actual donde apunta el puntero, asignando dichos valores a variables locales locales previamente declaradas.
4. **CLOSE:** Libera la memoria consumida por el cursor. Si se omite, MySQL cerrará el cursor automáticamente al finalizar la rutina.

Ejemplo (Actualización masiva de Límite de Crédito basado en la actividad real del cliente):

Implementaremos un procedimiento que audite a todos los clientes de **Jardinería** mediante un cursor procedimental. Si un cliente ha acumulado en pagos efectivos más del 80% de su límite de crédito actual, el SGBD ampliará proactivamente su límite crediticio un 25%.

```
DELIMITER //
DROP PROCEDURE IF EXISTS sp_optimizar_credito_clientes_activos//
CREATE PROCEDURE sp_optimizar_credito_clientes_activos()
BEGIN
-- 1. Declarar variables de control interno
DECLARE v_finalizado INT DEFAULT FALSE;
DECLARE v_codigo_cliente INT;
DECLARE v_limite_actual DECIMAL(15,2);
DECLARE v_total_pagos DECIMAL(15,2);
-- 2. Declaración del cursor sobre todos los clientes activos con
crédito
DECLARE cursor_clientes CURSOR FOR
SELECT codigo_cliente, limite_credito
FROM cliente
WHERE limite_credito > 0.00;
-- 3. Handler asociado para controlar cuándo el cursor se queda sin
filas (NOT FOUND)
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_finalizado = TRUE;
-- 4. Abrir la consulta y cargar el puntero
OPEN cursor_clientes;
bucle_extraccion: LOOP
-- 5. Extraer registro actual
FETCH cursor_clientes INTO v_codigo_cliente, v_limite_actual;
-- Control del escape del bucle
IF v_finalizado THEN
LEAVE bucle_extraccion;
END IF;
-- 6. Ejecutar lógica procedural para evaluar la regla de negocio
SELECT COALESCE(SUM(total), 0.00) INTO v_total_pagos
FROM pago
WHERE codigo_cliente = v_codigo_cliente;
-- Si el total de pagos reales acumulados supera el 80% de su límite,
aumentamos el límite
IF v_total_pagos > (v_limite_actual * 0.80) THEN
UPDATE cliente
SET limite_credito = v_limite_actual * 1.25
WHERE codigo_cliente = v_codigo_cliente;
END IF;
END LOOP bucle_extraccion;
```

```
-- 7. Cerrar cursor para liberar recursos de red
CLOSE cursor_clientes;
END//
DELIMITER ;
-- Ejecutamos la reevaluación de límites del sistema completo de
Jardinería
CALL sp_optimizar_credito_clientes_activos();
```

7. DISPARADORES (TRIGGERS)

Un disparador o *trigger* es un objeto de base de datos asociado a una tabla que se activa de manera automática e implícita cuando se ejecuta un evento DML de modificación de datos (INSERT, UPDATE o DELETE) sobre dicha tabla [1.8.8].

7.1 Lógica de ejecución: BEFORE/AFTER e INSERT/UPDATE/DELETE

Un disparador puede configurarse para activarse en dos instantes temporales distintos:

- **BEFORE:** Se ejecuta antes de que los cambios se validen e intenten almacenar en el disco del servidor. Es la opción idónea para realizar operaciones de validación, saneamiento de datos de entrada o cálculo de campos calculados [1.8.8].
- **AFTER:** Se ejecuta una vez que los cambios han sido aplicados con éxito en la tabla de datos. Es ideal para mantener registros de auditoría o históricos en tablas paralelas [1.8.8].

7.2 Pseudotablas OLD y NEW y su aplicación en la integridad de datos

Dependiendo del tipo de instrucción DML ejecutada, el cuerpo del trigger tiene acceso a los valores mediante las variables contextuales:

- ```
  NEW: Objeto temporal que contiene los valores de las
  columnas que se están insertando o modificando. (Disponible en INSERT y
  UPDATE) [1.8.8].
  ```
  ```

- **OLD:** Objeto temporal que contiene los valores de las columnas previos a la actualización o eliminación. (Disponible en UPDATE y DELETE) [1.8.8].

Operación	Acceso a NEW	Acceso a OLD
INSERT	Sí (Lectura / Modificación en BEFORE)	No
UPDATE	Sí (Lectura / Modificación en BEFORE)	Sí (Solo Lectura)
DELETE	No	Sí (Solo Lectura)

Ejemplo de Trigger BEFORE INSERT (Validación de stock e integridad referencial):

Un SysAdmin debe impedir que se registre un pedido de productos de jardinería si no existe stock real suficiente en el inventario, previniendo incoherencias comerciales.

```
DELIMITER //
DROP TRIGGER IF EXISTS trigger_validar_stock_before_insert//
CREATE TRIGGER trigger_validar_stock_before_insert
BEFORE INSERT ON detalle_pedido
FOR EACH ROW
BEGIN
DECLARE v_stock_disponible INT;
-- Obtener la cantidad de existencias físicas reales del producto
seleccionado
SELECT cantidad_en_stock INTO v_stock_disponible
FROM producto
WHERE codigo_producto = NEW.codigo_producto;
-- Si se solicita mayor cantidad que la disponible, generamos una
excepción personalizada
IF NEW.cantidad > v_stock_disponible THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'OPERACIÓN RECHAZADA: Stock insuficiente para
procesar la línea del producto.';
END IF;
END//
DELIMITER ;
```

Ejemplo de Trigger AFTER INSERT (Auditoría e Históricos automáticos):

Implementaremos un disparador que nos permita registrar en tiempo real un log de pagos que van realizando los clientes. Primero creamos una tabla dedicada a centralizar las

notificaciones de pagos [1.10].

```
eroría-- Creamos la tabla de soporte técnico de notificaciones
CREATE TABLE IF NOT EXISTS notificaciones (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
total DECIMAL(15,2) NOT NULL,
codigo_cliente INT NOT NULL
);
DELIMITER //
DROP TRIGGER IF EXISTS trigger_notificar_pago_after_insert//
CREATE TRIGGER trigger_notificar_pago_after_insert
AFTER INSERT ON pago
FOR EACH ROW
BEGIN
-- Almacena de manera automática el pago registrado en nuestra tabla
de notificaciones
INSERT INTO notificaciones (total, codigo_cliente)
VALUES (NEW.total, NEW.codigo_cliente);
END//
DELIMITER ;
```

8. ENLACES DE REFERENCIA Y DOCUMENTACIÓN OFICIAL

Para profundizar en la sintaxis procedimental avanzada, gestión de pilas de llamadas y optimización de ejecución del motor de rutinas almacenadas en MySQL 8.0, consulta los siguientes recursos del manual técnico oficial:

- MySQL 8.0 Reference Manual: Stored Objects (Procedures & Functions)
- MySQL 8.0 Reference Manual: Trigger Syntax and Examples
- MySQL 8.0 Reference Manual: Flow Control Statements
- MySQL 8.0 Reference Manual: Cursors implementation
- MySQL 8.0 Reference Manual: Condition Handling and SIGNAL SQLSTATE